Test 2 — Part B

Trace & write code · 20 points — ANSWER KEY

B1 Write the Code: Truck

6 pts

Write a class `Truck` that publicly inherits from `Vehicle` below. A Truck has a `double payload` (in tons) and always has 6 wheels. Give it a constructor `Truck(string m, double p)` that **chains** to the Vehicle constructor, and **override** `describe()` so it prints the Vehicle line first (by calling the base version) and then the payload.

GIVEN

```
class Vehicle {
protected:
    string make;
    int wheels;
public:
    Vehicle(string m, int w) : make(m), wheels(w) {}
    virtual ~Vehicle() {}
    virtual void describe() const {
        cout << make << " with " << wheels << " wheels" << endl;
    }
};
```

USAGE & EXPECTED OUTPUT

```
Truck t("Volvo", 12.5);
t.describe();
// Volvo with 6 wheels
// Payload: 12.5 tons
```

MODEL ANSWER

```
class Truck : public Vehicle {
    double payload;
public:
    Truck(string m, double p) : Vehicle(m, 6), payload(p) {}
    void describe() const override {
        Vehicle::describe();
        cout << "Payload: " << payload << " tons" << endl;
    }
};
```

RUBRIC · 1 PT EACH

1	`class Truck : public Vehicle` (no `public` = private inheritance, lose this point)
1	`double payload;` as a data member
1	`Vehicle(m, 6)` in the initializer list. Assigning `make` / `wheels` in the body does not compile (no `Vehicle()`), so no point.
1	`payload(p)` initialized (list or body both OK)
1	`describe() const` with matching signature (`override` keyword recommended; missing `const` = hides, lose point)
1	Calls `Vehicle::describe();` then prints the payload. Plain `describe();` recurses forever, lose point.

Test 2 — Part B

Trace & write code · 20 points — ANSWER KEY

B2 One Loop, Many Types

8 pts · 1 per cell

Fill in exactly what each call prints for each object in `crew`. If a class doesn't override a function, think about which version it falls back to.

```
class Robot {
protected:
    string id;
public:
    Robot(string i) : id(i) {}
    virtual ~Robot() {}
    virtual void greet() const { cout << id << ": beep" << endl; }
    virtual void work() const  { cout << id << " idles" << endl; }
};
class Welder : public Robot {
public:
    Welder(string i) : Robot(i) {}
    void greet() const override { cout << "Welder " << id << " online" << endl; }
    void work() const override  { cout << id << " welds a seam" << endl; }
};
class Painter : public Robot {
public:
    Painter(string i) : Robot(i) {}
    void work() const override { cout << id << " paints a wall" << endl; }
};
class Butler : public Robot {
public:
    Butler(string i) : Robot(i) {}
    void greet() const override {
        Robot::greet();
        cout << "  (" << id << " bows)" << endl;
    }
};

int main() {
    vector<unique_ptr<Robot>> crew;
    crew.push_back(make_unique<Welder>("W1"));
    crew.push_back(make_unique<Painter>("P2"));
    crew.push_back(make_unique<Butler>("B3"));
    crew.push_back(make_unique<Robot>("R4"));
    for (auto& r : crew) {
        r->greet();
        r->work();
    }
}
```

OBJECT	R->GREET() PRINTS	R->WORK() PRINTS
Welder("W1")	`Welder W1 online`	`W1 welds a seam`
Painter("P2")	`P2: beep`	`P2 paints a wall`
Butler("B3")	`B3: beep` `  (B3 bows)`	`B3 idles`
Robot("R4")	`R4: beep`	`R4 idles`

GRADING NOTES

1 pt per cell. Painter's greet and Butler's work fall back to the inherited `Robot` version; those two cells are the point of the question. Butler's greet needs both lines for the point. Ignore spacing differences.

Test 2 — Part B

Trace & write code · 20 points — ANSWER KEY

B3 Write the Code: Abstract Shapes — 6 pts

Use this abstract `Shape` class:

```
class Shape {
protected:
    string label;
public:
    Shape(string l) : label(l) {}
    virtual ~Shape() {}
    virtual double perimeter() const = 0;       // pure virtual
    string getLabel() const { return label; }
};
```

(a) Write a **concrete** class `Triangle` with three `double` sides, a constructor `Triangle(double x, double y, double z)` that chains `Shape("triangle")`, and a `perimeter()` that returns the sum of the sides.

(b) Write `int countBig(const vector<Shape*>& shapes, double limit)` that returns how many shapes have a perimeter greater than `limit`. It must work for any Shape, including ones written next year.

(A) MODEL ANSWER

```
class Triangle : public Shape {
    double a;
    double b;
    double c;
public:
    Triangle(double x, double y, double z)
        : Shape("triangle"), a(x), b(y), c(z) {}
    double perimeter() const override {
        return a + b + c;
    }
};
```

(B) MODEL ANSWER

```
int countBig(const vector<Shape*>& shapes, double limit) {
    int count = 0;
    for (Shape* s : shapes) {
        if (s->perimeter() > limit) {
            count++;
        }
    }
    return count;
}
```

Check: `Square(2)`, `Square(5)`, `Triangle(3,4,5)` with limit 10 → perimeters 8, 20, 12 → returns 2.

RUBRIC · 1 PT EACH

1	(a) `class Triangle : public Shape` with three `double` sides
1	(a) Constructor chains `Shape("triangle")` and sets the sides
1	(a) `double perimeter() const` returns `a + b + c`. Missing `const` leaves Triangle abstract: 0 for this line.
1	(b) Loops over every pointer in `shapes`
1	(b) Calls `s->perimeter()` through the base pointer and compares to `limit`. Mentioning `Square` / `Triangle` or casting: 0.
1	(b) Counts and returns an `int`